Quantum Computing

Dirac Notation

Grading

Post the output of the following code into your document:

```
myState2=[
   (numpy.sqrt(0.1)*1.j, '101'),
   (numpy.sqrt(0.5), '000') ,
   (-numpy.sqrt(0.4), '010' )]
 PrettyPrintBinary(myState2)
 PrettyPrintInteger(myState2)
```

```
print(DiracToVec(myState2))
print(VecToDirac(DiracToVec(myState2)))
```

Paste the result of running your code on the above output:

[np.complex128(0.7071067811865476+0j), np.complex128(0j),
np.complex128(-0.6324555320336759+0j), np.complex128(0j), np.complex128(0j),
np.complex128(0.31622776601683794j), np.complex128(0j), np.complex128(0j)]
[(np.complex128(0.7071067811865476+0j), '000'), (np.complex128(0j), '001'),
(np.complex128(-0.6324555320336759+0j), '010'), (np.complex128(0j), '011'),
(np.complex128(0j), '100'), (np.complex128(0.31622776601683794j), '101'),
(np.complex128(0j), '110'), (np.complex128(0j), '111')]

QC Simulators

Simulator S

Paste the result from **rand.circuit** for Simulator S

[(np.complex128(0.14565796004368065+0.11348294333323074j), '00000'),
(np.complex128(-0.00017826157500451523+0.08005459383129264j), '00010'),
(np.complex128(0.001474690001990256-0.036459032126468345j), '10000'),
(np.complex128(-0.08205100536822746+0.015513990179495646j), '10010'),
(np.complex128(0.043097550946832736-0.12184596055817097j), '00001'),
(np.complex128(0.05035629868818656-0.007282489799443279j), '00011'),
(np.complex128(-0.012673925486559965+0.005514371048822509j), '10001'),
(np.complex128(0.016011739671690253+0.02259413933738235j), '10011'),
(np.complex128(0.07232672559216355-0.02261927030390635j), '00100'),
(np.complex128(0.05394795241192365+0.008954381028197886j), '00110'),
(np.complex128(-0.17165615534648057+0.366922113805503j), '10100'),
(np.complex128(-0.14693154505266529-0.04361461090635174j), '10110'),
(np.complex128(-0.03444784817538661-0.042228210929983696j), '00101'),
(np.complex128(0.0017934458673293414-0.026754015134267246j), '00111'),
(np.complex128(0.2774713578139312+0.04232940722894598j), '10101'),
(np.complex128(0.007401893528164838+0.10788583514551447j), '10111'),
(np.complex128(0.13464628848368854-0.12560319656579141j), '01000'),
(np.complex128(0.010652323173449801+0.07094564243091285j), '01010'),
(np.complex128(-0.4426796395773186-0.08917051669782353j), '11000'),
(np.complex128(-0.054788575831034844-0.21473166615430533j), '11010'),
(np.complex128(-0.1040997208563847-0.0676238124186473j), '01001'),
(np.complex128(0.031536789559932346-0.035377797720121625j), '01011'),
(np.complex128(0.03432507377519414+0.31797145079525085j), '11001'),
(np.complex128(-0.12098548283141342+0.06544840304773131j), '11011'),
(np.complex128(0.007920474449196635+0.2978985122057255j), '01100'),
(np.complex128(-0.1130653010103075-0.022843082802562545j), '01110'),
(np.complex128(-0.18144336972286915-0.07802773562153237j), '11100'),
(np.complex128(-0.05781065216328528+0.01012335649380973j), '11110'),
(np.complex128(0.19608126814268645-0.05835344971980602j), '01101'),
(np.complex128(0.025670653436355977+0.07563944999428013j), '01111'),
(np.complex128(-0.005850147569994732+0.12703336730376008j), '11101'),
(np.complex128(-0.011813081237055323+0.020044171373334262j), '11111')]

((0.14565796004368065+0.11348294333323074j) | 00000 > +

(-0.00017826157500451523+0.08005459383129264j) | 00010 > +
(0.001474690001990256-0.036459032126468345j) | 10000 > +
(-0.08205100536822746+0.015513990179495646j) | 10010 > +
(0.043097550946832736-0.12184596055817097j) | 00001 > +
(0.05035629868818656-0.007282489799443279j) | 00011 > +
(-0.012673925486559965+0.005514371048822509j) | 10001 > +
(0.016011739671690253+0.02259413933738235j) | 10011 > +
(0.07232672559216355-0.02261927030390635j) | 00100 > +
(0.05394795241192365+0.008954381028197886j) | 00110 > +
(-0.17165615534648057+0.366922113805503j) | 10100 > +
(-0.14693154505266529-0.04361461090635174j) | 10110 > +
(-0.03444784817538661-0.042228210929983696j) | 00101 > +
(0.0017934458673293414-0.026754015134267246j) | 00111 > +
(0.2774713578139312+0.04232940722894598j) | 10101 > +
(0.007401893528164838+0.10788583514551447j) | 10111 > +
(0.13464628848368854-0.12560319656579141j) | 01000 > +
(0.010652323173449801+0.07094564243091285j) | 01010 > +
(-0.4426796395773186-0.08917051669782353j) | 11000 > +
(-0.054788575831034844-0.21473166615430533j) | 11010 > +
(-0.1040997208563847-0.0676238124186473j) | 01001 > +
(0.031536789559932346-0.035377797720121625j) | 01011 > +
(0.03432507377519414+0.31797145079525085j) | 11001 > +
(-0.12098548283141342+0.06544840304773131j) | 11011 > +
(0.007920474449196635+0.2978985122057255j) | 01100 > +
(-0.1130653010103075-0.022843082802562545j) | 01110 > +
(-0.18144336972286915-0.07802773562153237j) | 11100 > +
(-0.05781065216328528+0.01012335649380973j) | 11110 > +
(0.19608126814268645-0.05835344971980602j) | 01101 > +
(0.025670653436355977+0.07563944999428013j) | 01111 > +
(-0.005850147569994732+0.12703336730376008j) | 11101 > +
(-0.011813081237055323+0.020044171373334262j) | 11111 >)

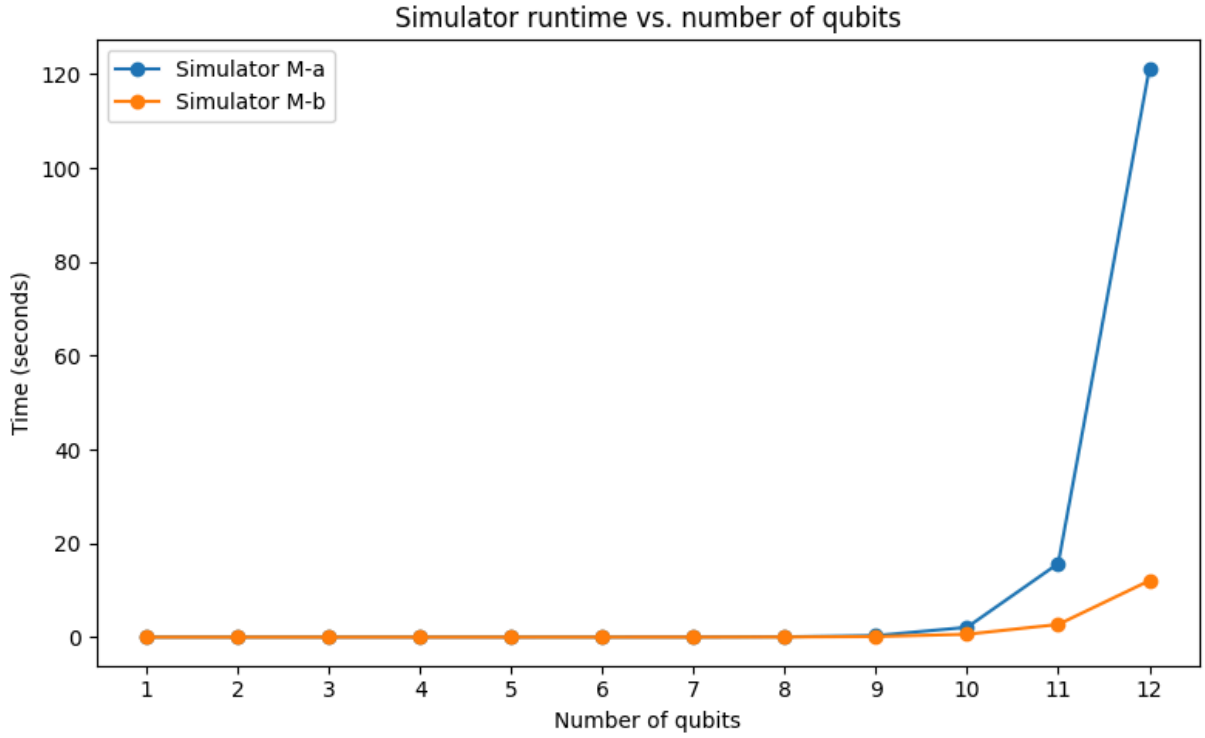
Quantum Simulator M

My simulator M results for the three circuit tests (should agree with previous results):

My simulator

- ❏ Check that simulator M-a gives the same results as Ia for the rand.circuit: yes
- ❏ Check that simulator M-b gives the same results as S for the rand.circuit: yes
- ❏ Check that simulator M-c gives the same results as S for the rand.circuit (extra credit)
- ❏ Check that simulator M-d gives the same results as S for the rand.circuit (extra credit)

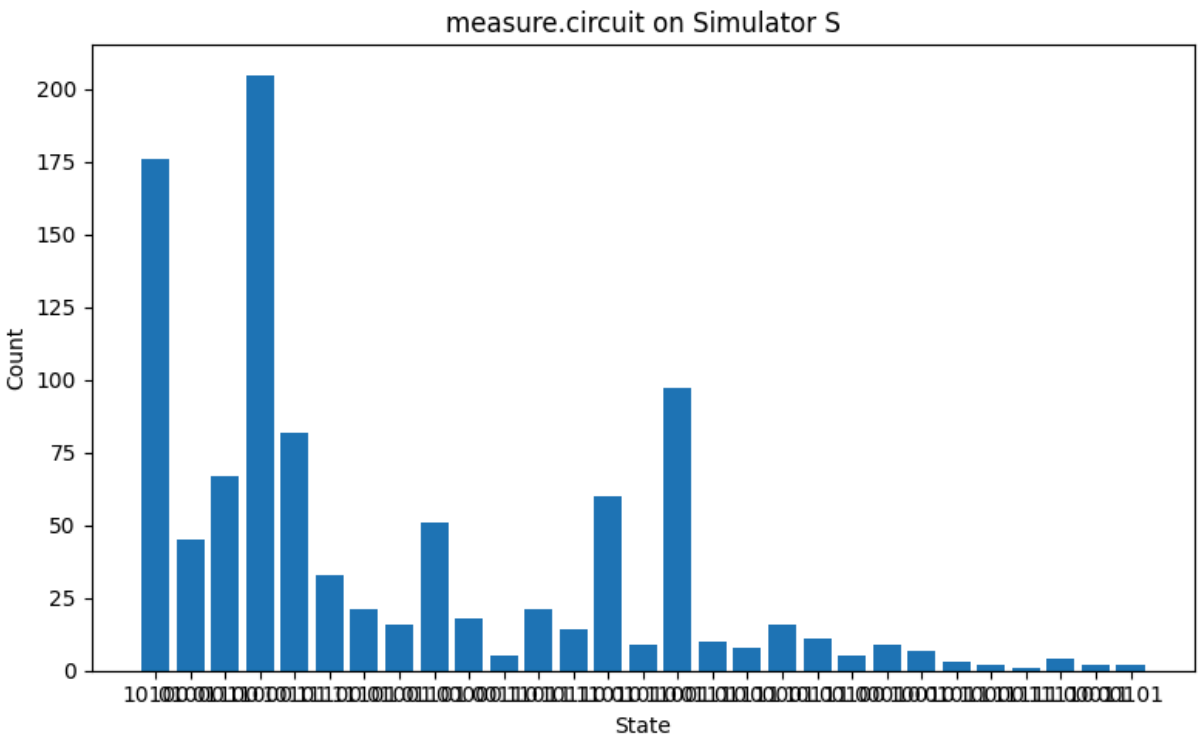
Paste the output of your time and RAM tests for the simulators you have implemented.




```
n= 1  M-a:    0.01 MB   M-b:    0.01 MB
n= 2  M-a:    0.01 MB   M-b:    0.01 MB
n= 3  M-a:    0.01 MB   M-b:    0.01 MB
n= 4  M-a:    0.03 MB   M-b:    0.02 MB
n= 5  M-a:    0.09 MB   M-b:    0.06 MB
n= 6  M-a:    0.32 MB   M-b:    0.22 MB
n= 7  M-a:    1.12 MB   M-b:    0.60 MB
n= 8  M-a:    3.68 MB   M-b:    1.59 MB
n= 9  M-a:   14.69 MB   M-b:    6.32 MB
n=10  M-a:   58.74 MB   M-b:   25.22 MB
```

Measuring and Input

Paste the histogram from doing **measure.circuit** on simulator S



Paste the output from **input.circuit** and **myInputState.txt** for Simulator S

Simulator S result:

((0.15409919900412572-0.039023142873475775j) | 00000 > +
(-0.02131981307535495+0.08774787414090221j) | 00010 > +
(-0.1480464106395854+0.08745635006745286j) | 10000 > +
(0.11691184507936053+0.013808709741378308j) | 10010 > +
(0.04895105577891031-0.03001851549580141j) | 00001 > +
(-0.1116091892870344-0.23332812293116612j) | 00011 > +
(-0.06328890454391892+0.25662035423302676j) | 10001 > +
(-0.02667646151394929-0.004062838923814427j) | 10011 > +
(-0.033901258348144725-0.009674003342578126j) | 00100 > +
(-0.19592098529914306+0.009885177232733758j) | 00110 > +
(0.21923132553221591-0.21304182634262642j) | 10100 > +
(-0.1479391543721399-0.10481145122901085j) | 10110 > +

(-0.016746063710111725-0.13026586952891356j) | 00101 > +
(-0.09796097290238416+0.013475996833847481j) | 00111 > +
(-0.048941996178234734-0.11635508158684055j) | 10101 > +
(0.2231675743280913+0.05083463296709693j) | 10111 > +
(-0.18082511268453239+0.01438104561772749j) | 01000 > +
(0.0010329113421169653-0.0034553374325645703j) | 01010 > +
(-0.13963947717339315-0.23720129948549148j) | 11000 > +
(-0.2632220901419421-0.0656005073251216 3j) | 11010 > +
(0.05382740960549368-0.06863464649457765j) | 01001 > +
(-0.02154097912106914+0.01744613535918265j) | 01011 > +
(0.1847507110418059-0.005148336320294392j) | 11001 > +
(0.17519678454879786-0.02010633083183433j) | 11011 > +
(-0.007195884290384175+0.15958103150789765j) | 01100 > +
(-0.03789377980145897+0.005859520326027703j) | 01110 > +
(0.11220121663454645+0.0012498444406552484j) | 11100 > +
(-0.20830028473401127+0.07702445377195208j) | 11110 > +
(-0.22309649987486385+0.0746289884182196j) | 01101 > +
(0.07390340399259629-0.05074229743090867j) | 01111 > +
(0.024190828060211902-0.12445058311966187j) | 11101 > +
(0.05476980548434894+0.306130439585481 7j) | 11111 >)

Non-Atomic Gates

Circuit Description for Not:

NOT: [['H', '0'], ['P', '0', '3.141592653589793'], ['H', '0']]

Result of Not on |1>:

((0.9999999999999998-6.123233995736765e-17j) | 0 >)

Circuit Description for Rz:
Rz: [['P', '0', '0.5']]

Circuit Description for (short-range) Control-Rz:
CRz: [['P', '1', '0.25'], ['CNOT', '0', '1'], ['P', '1', '-0.25'], ['CNOT', '0', '1']]

Circuit Description for (short-range) Control-Phase:

CPhase: [['P', '0', '0.25'], ['P', '1', '0.25'], ['CNOT', '0', '1'], ['P', '1', '-0.25'], ['CNOT', '0', '1']]

Circuit Description for Swap(2,5) (using short-range gates):
Swap(2,5): [['CNOT', '2', '5'], ['CNOT', '5', '2'], ['CNOT', '2', '5']]

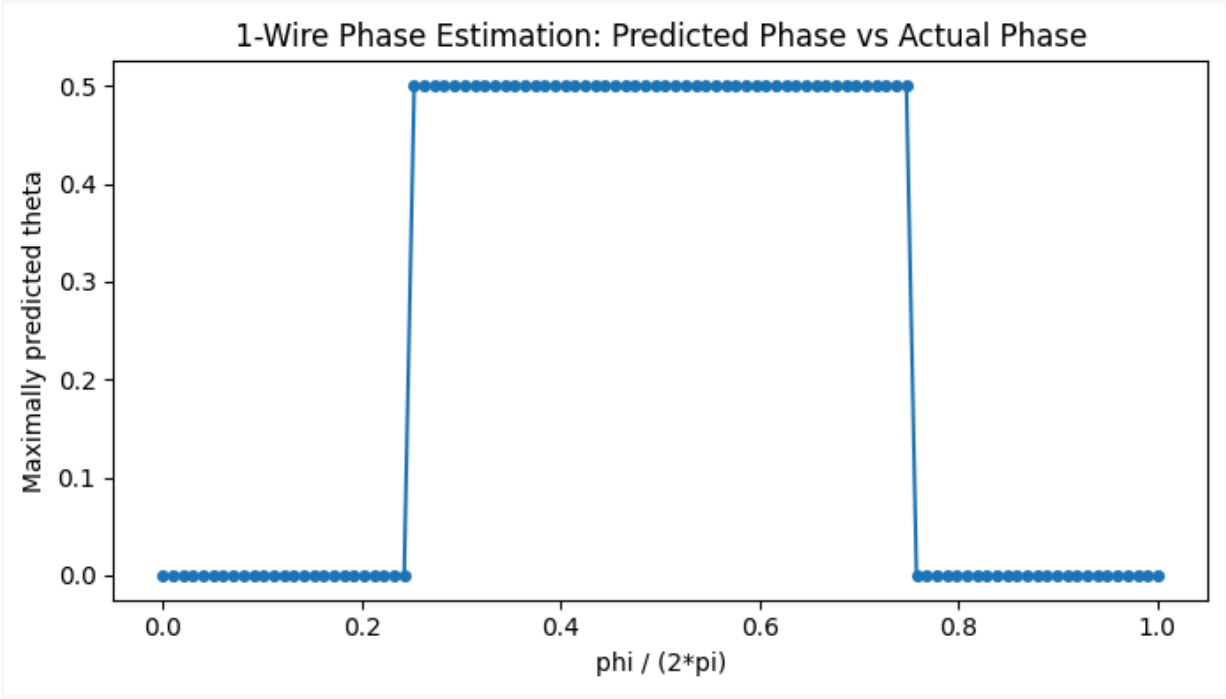
9
H 0
CPHASE 0 5 0.3
P 1 0.3
CNOT 4 7
SWAP 2 8

Result of running your circuit (after precompilation) on the above input:

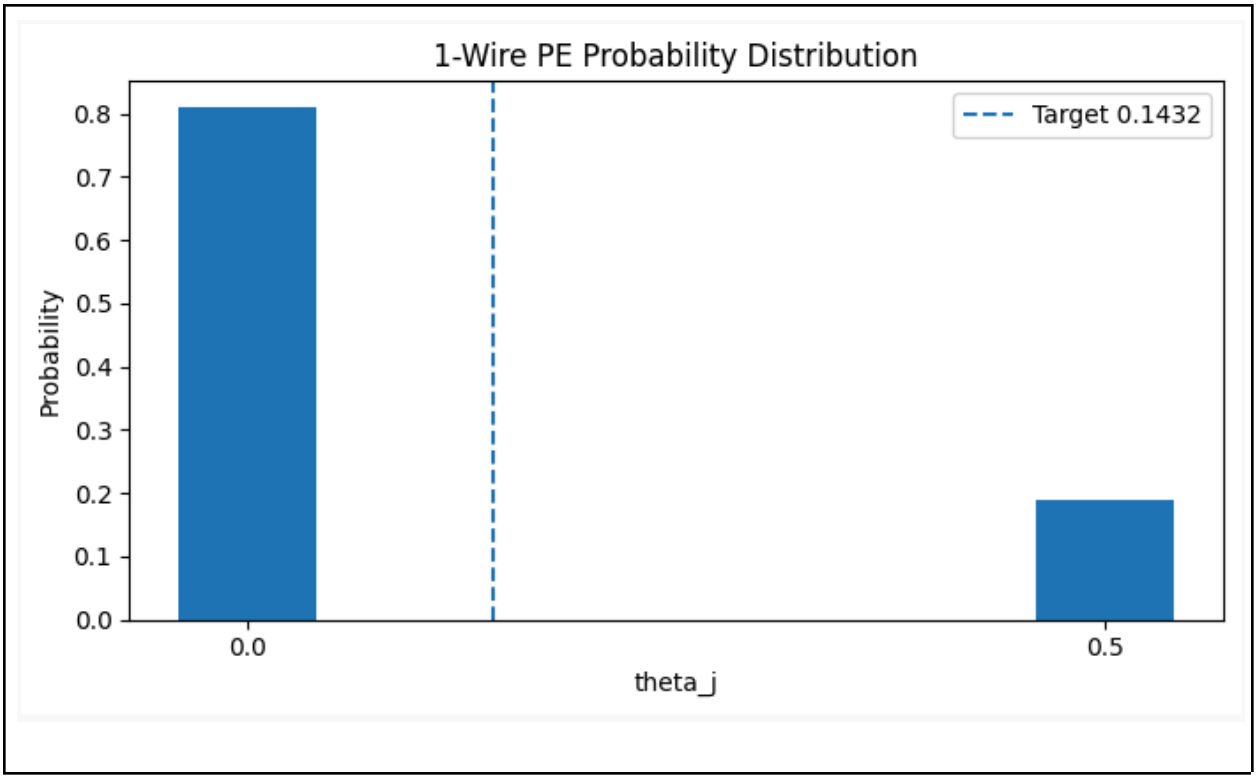
9
INITSTATE BASIS |000000000>
H 0
P 0 0.15
P 5 0.15
CNOT 0 5
P 5 -0.15
CNOT 0 5
P 1 0.3
CNOT 4 7
CNOT 2 8
CNOT 8 2
CNOT 2 8

Phase Estimation

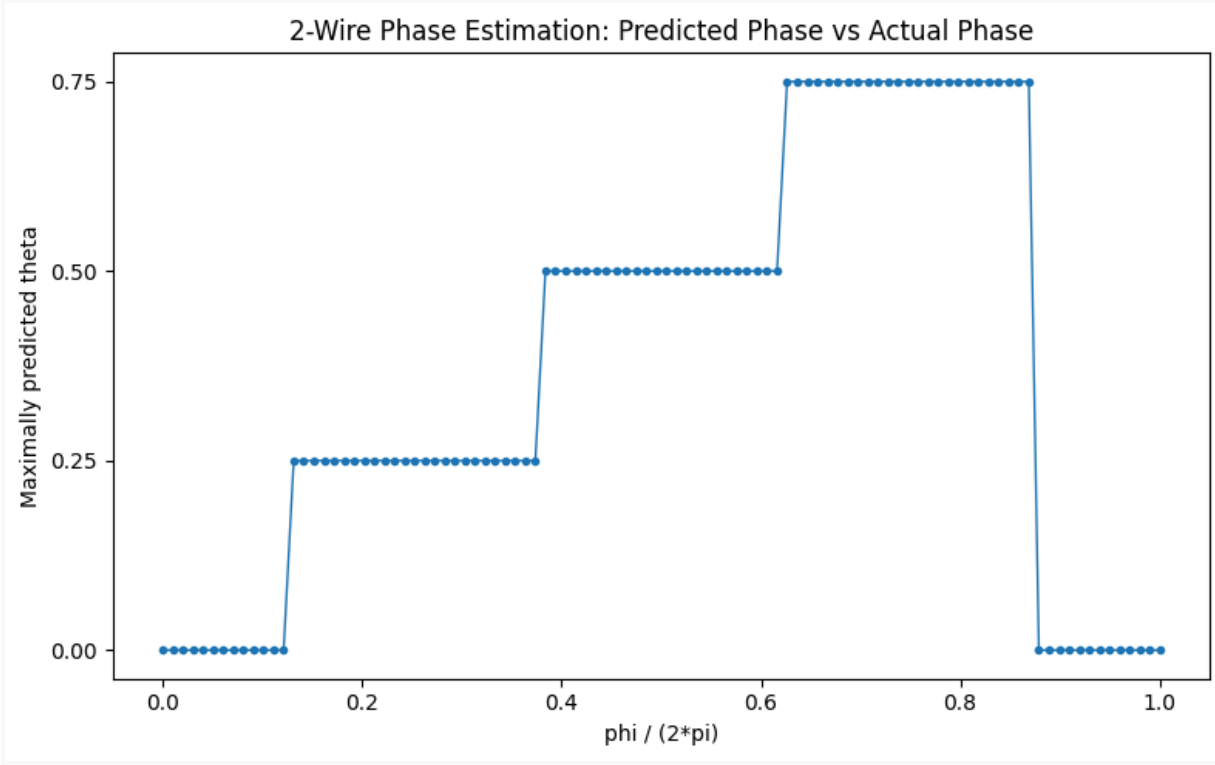
1 Wire: Run your simulator for 100 evenly chosen values of ϕ between 0 and 2pi and make the following graph: On the x-axis put ϕ/(2π) and on the y-axis put the θj maximally predicted by your circuit

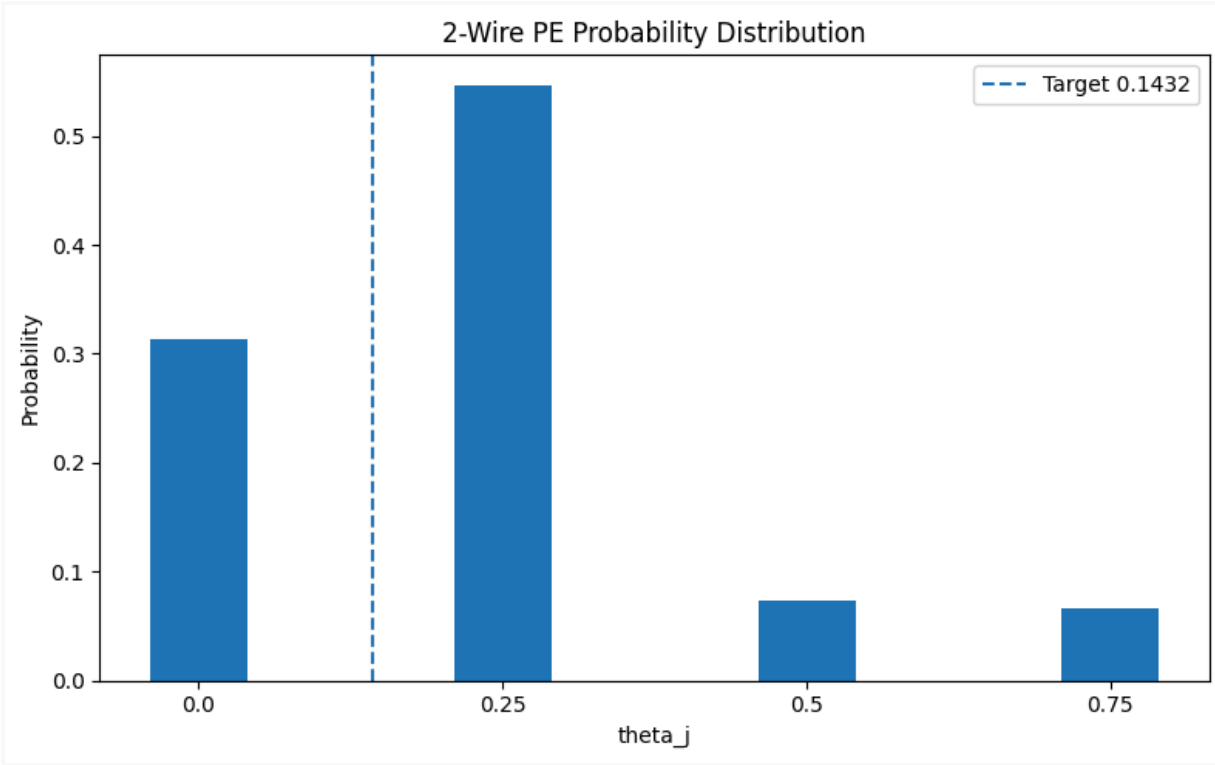


1 Wire: As a separate graph, let $\phi/(2\pi)=0.1432394487827058$ and graph a histogram of the probability your circuit gives back the result θ_j (as a function of θ_j). Paste your histogram and mark on your histogram 0.1432:

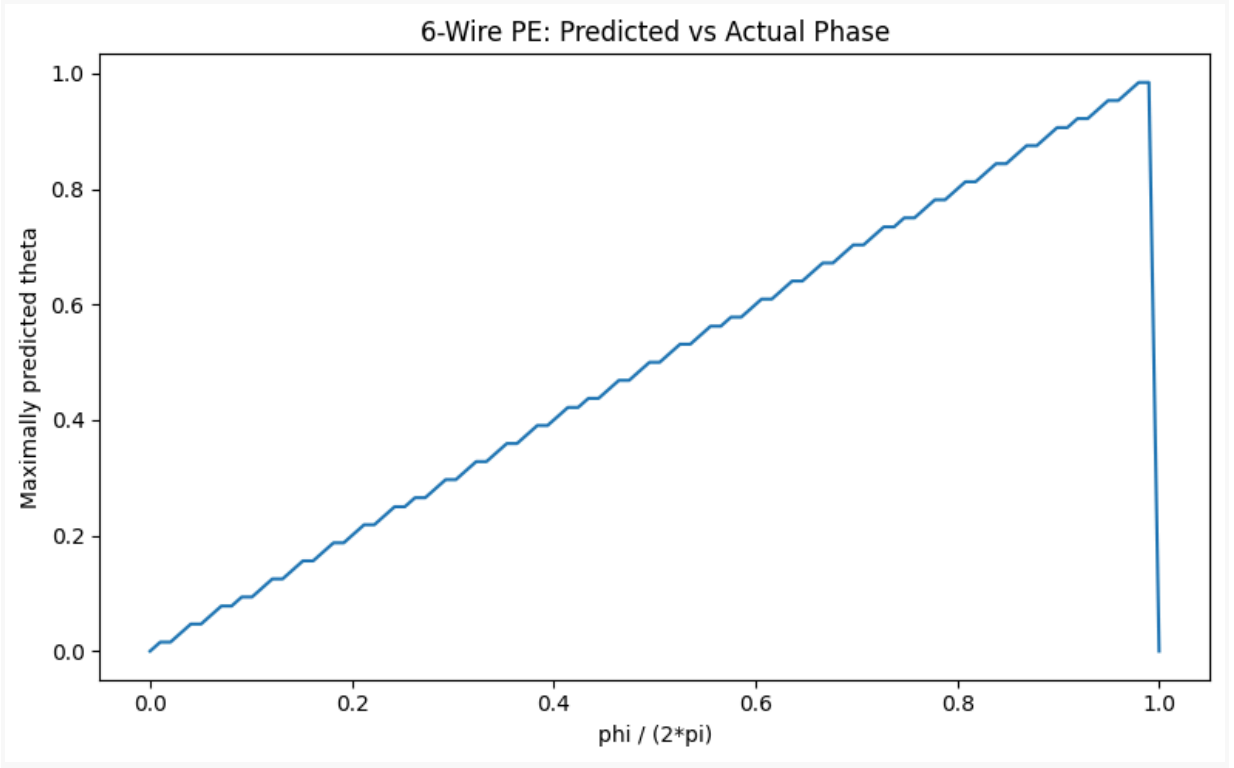


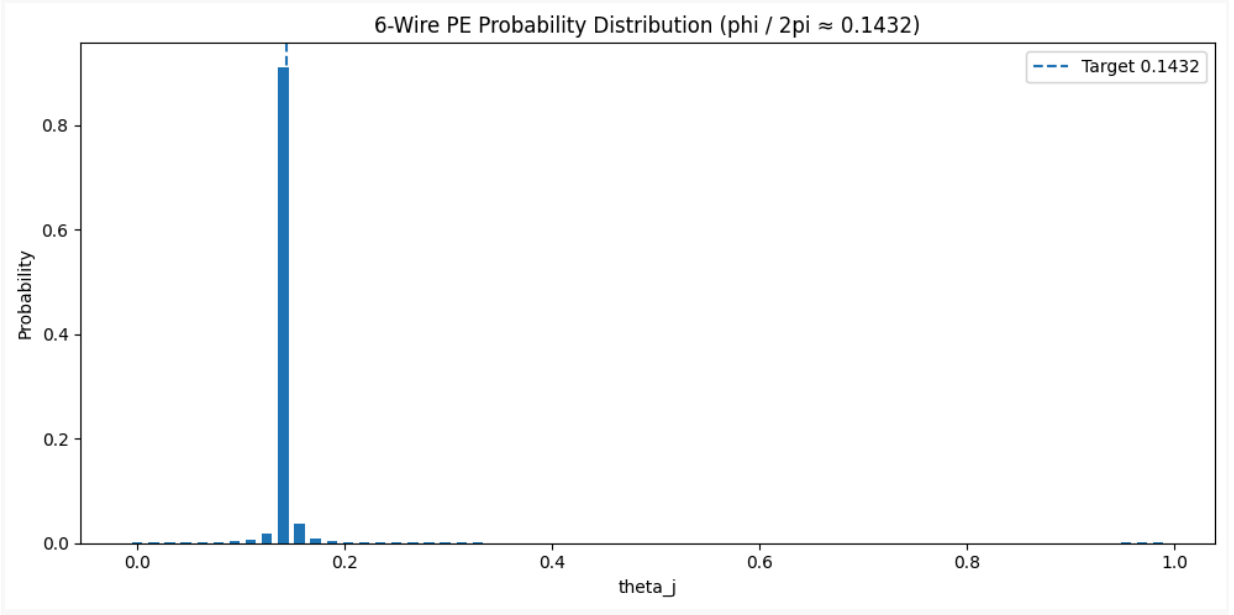
Produce the same graphs with **2 wires** on top:


2-Wire Phase Estimation: Predicted Phase vs Actual Phase
Maximally predicted theta
0.00
0.25
0.50
0.75
0.0
0.2
0.4
0.6
0.8
1.0
phi / (2*pi)


2-Wire PE Probability Distribution
Target 0.1432
Probability
0.0
0.1
0.2
0.3
0.4
0.5
0.0
0.25
0.5
0.75
theta_j

Produce the same graphs with **6 wires** on top. Also paste the circuit description for this phase estimate circuit.





Paste a circuit description for the ϕ=0.5, 6 top wire phase estimation circuit that uses fewer gates to represent the Quantum Fourier Transform:

```
7
INITSTATE BASIS |0000001>
H 0
H 1
H 2
H 3
H 4
H 5
CPHASE 0 6 3.4336293856408275
CPHASE 1 6 1.7168146928204138
CPHASE 2 6 4.0
CPHASE 3 6 2.0
CPHASE 4 6 1.0
CPHASE 5 6 0.5
```

Come up with your own circuit description for phase estimation with a U on the bottom wire that is made of NOTs and P gates, rather than just a single phase gate as we have been doing. Run your phase estimation circuit with this U gate and generate a histogram of the possible outputs for the top wires.

```
7
INITSTATE BASIS |0000001>
H 0
H 1
H 2
H 3
H 4
H 5
CNOT 0 6
CPHASE 0 6 1.0471975511965976
CNOT 0 6
CPHASE 0 6 1.0471975511965976
CNOT 0 6
CPHASE 0 6 1.0471975511965976
CNOT 0 6
```

```
CPHASE 0 6 1.0471975511965976
CNOT 0 6
CPHASE 0 6 1.0471975511965976
CNOT 0 6
CPHASE 0 6 1.0471975511965976
CNOT 0 6
```

```
CNOT 0 6
CPHASE 0 6 1.0471975511965976
CNOT 0 6
CPHASE 0 6 1.0471975511965976
CNOT 0 6
CPHASE 0 6 1.0471975511965976
CNOT 0 6
CPHASE 0 6 1.0471975511965976
CNOT 0 6
CPHASE 0 6 1.0471975511965976
CNOT 1 6
CPHASE 1 6 1.0471975511965976
CNOT 1 6
CPHASE 1 6 1.0471975511965976
CNOT 1 6
CPHASE 1 6 1.0471975511965976
CNOT 1 6
CPHASE 1 6 1.0471975511965976
CNOT 1 6
CPHASE 1 6 1.0471975511965976
CNOT 1 6
CPHASE 1 6 1.0471975511965976
CNOT 1 6
CPHASE 1 6 1.0471975511965976
CNOT 1 6
CPHASE 1 6 1.0471975511965976
CNOT 1 6
CPHASE 1 6 1.0471975511965976
CNOT 1 6
CPHASE 1 6 1.0471975511965976
CNOT 1 6
CPHASE 1 6 1.0471975511965976
CNOT 1 6
CPHASE 1 6 1.0471975511965976
CNOT 1 6
CPHASE 1 6 1.0471975511965976
CNOT 1 6
CPHASE 1 6 1.0471975511965976
CNOT 1 6
CPHASE 1 6 1.0471975511965976
CNOT 1 6
CPHASE 1 6 1.0471975511965976
CNOT 1 6
CPHASE 1 6 1.0471975511965976
CNOT 1 6
CPHASE 1 6 1.0471975511965976
CNOT 2 6
CPHASE 2 6 1.0471975511965976
CNOT 2 6
CPHASE 2 6 1.0471975511965976
CNOT 2 6
```

```
CPHASE 2 6 1.0471975511965976
CNOT 2 6
CPHASE 2 6 1.0471975511965976
CNOT 2 6
CPHASE 2 6 1.0471975511965976
CNOT 2 6
CPHASE 2 6 1.0471975511965976
CNOT 2 6
CPHASE 2 6 1.0471975511965976
CNOT 2 6
CPHASE 2 6 1.0471975511965976
CNOT 3 6
CPHASE 3 6 1.0471975511965976
CNOT 3 6
CPHASE 3 6 1.0471975511965976
CNOT 3 6
CPHASE 3 6 1.0471975511965976
CNOT 3 6
CPHASE 3 6 1.0471975511965976
CNOT 4 6
CPHASE 4 6 1.0471975511965976
CNOT 4 6
CPHASE 4 6 1.0471975511965976
CNOT 5 6
CPHASE 5 6 1.0471975511965976
H 0
CPHASE 0 1 -1.5707963267948966
CPHASE 0 2 -0.7853981633974483
CPHASE 0 3 -0.39269908169872414
CPHASE 0 4 -0.19634954084936207
CPHASE 0 5 -0.09817477042468103
H 1
CPHASE 1 2 -1.5707963267948966
CPHASE 1 3 -0.7853981633974483
CPHASE 1 4 -0.39269908169872414
CPHASE 1 5 -0.19634954084936207
H 2
CPHASE 2 3 -1.5707963267948966
CPHASE 2 4 -0.7853981633974483
CPHASE 2 5 -0.39269908169872414
H 3
CPHASE 3 4 -1.5707963267948966
CPHASE 3 5 -0.7853981633974483
H 4
CPHASE 4 5 -1.5707963267948966
H 5
SWAP 0 5
```

SWAP 1 4
SWAP 2 3

Quantum Fourier Transform

Paste the circuit description for the three-qubit Quantum Fourier Transform (QFT). Demonstrate that it produces the correct output when run through your simulator:

```
3
H 0
CPHASE 0 1 1.5707963267948966
CPHASE 0 2 0.7853981633974483
H 1
CPHASE 1 2 1.5707963267948966
H 2
SWAP 0 2
```

Paste the circuit description for the five-qubit QFT. Demonstrate that it produces the correct output when run through your simulator. Show the output of the five-qubit QFT when run with the myInputState input file:

```
5
H 0
CPHASE 0 1 1.5707963267948966
CPHASE 0 2 0.7853981633974483
CPHASE 0 3 0.39269908169872414
CPHASE 0 4 0.19634954084936207
H 1
CPHASE 1 2 1.5707963267948966
CPHASE 1 3 0.7853981633974483
CPHASE 1 4 0.39269908169872414
H 2
CPHASE 2 3 1.5707963267948966
CPHASE 2 4 0.7853981633974483
H 3
CPHASE 3 4 1.5707963267948966
H 4
SWAP 0 4
SWAP 1 3

( (-0.04028017284397896-0.22092218580979775j) | 00000 > +
(0.14110242611258708+0.16650808261166422j) | 00001 > +
(0.1847730989152713+0.0639044565767554j) | 00010 > +
```

(0.00028527132556335766-0.0222469854871524j) | 00011 > +
(0.15665937326444307-0.08463583856503606j) | 00100 > +
(0.21186219852653587-0.13549377193560447j) | 00101 > +
(0.024436563546173122+0.007291647371941323j) | 00110 > +
(0.12275573414814123-0.20487331415853316j) | 00111 > +
(0.21663304951566448+0.020112971083311827j) | 01000 > +
(0.1072369378950936+0.0013057823514517099j) | 01001 > +
(-0.2352333856256579-0.0009765323146217397j) | 01010 > +
(-0.07704606573094851-0.1673917459145033j) | 01011 > +
(-0.0014405690537449+0.10863479965142829j) | 01100 > +
(-0.04521301202662257-0.11330274327482456j) | 01101 > +
(-0.1991245557186332-0.11323570695048288j) | 01110 > +
(-0.15946600095304173+0.14668225282002403j) | 01111 > +
(-0.03374953559586265+0.07942275396910156j) | 10000 > +
(0.02799599698216207-0.0006560027147068798j) | 10001 > +
(-0.028812459116676015-0.08531366289419069j) | 10010 > +
(-0.2533515917977188-0.23087483698942998j) | 10011 > +
(0.24414416577410553+0.038853899872371374j) | 10100 > +
(-0.12503365026646246-0.007347522381533522j) | 10101 > +
(-0.14934362602958878-0.042653384487615034j) | 10110 > +
(0.17684418571117028-0.050584567395714986j) | 10111 > +
(-0.11810635653448785+0.01623973860915606j) | 11000 > +
(-0.10529823498224487+0.03863507247432339j) | 11001 > +
(-0.029289525816246692+0.012532684274186374j) | 11010 > +
(-0.2140564342692548+0.08433455953812156j) | 11011 > +
(0.12578799246104258-0.02452186818470413j) | 11100 > +
(-0.07697244593325292-0.12275736706850485j) | 11101 > +
(0.0763007320604176-0.11869394467289082j) | 11110 > +
(-0.17228309183862875+0.002035673115859865j) | 11111 >)

Classical Shors

Show that your code successfully factors numbers up to 10 bits:
N = 15 -> 5 * 3 = 15
N = 21 -> 7 * 3 = 21
N = 35 -> 7 * 5 = 35
N = 33 -> 3 * 11 = 33
N = 55 -> 5 * 11 = 55
N = 77 -> 11 * 7 = 77
N = 91 -> 7 * 13 = 91
N = 143 -> 11 * 13 = 143

N = 221 -> 13 * 17 = 221
N = 323 -> 19 * 17 = 323
N = 989 -> 23 * 43 = 989

Factor the number 33 and give the x and r you find:
N = 33 -> 3 * 11, x = 28, r = 10
N = 33 -> 11 * 3, x = 14, r = 10
N = 33 -> 3 * 11, x = 19, r = 10

Put a list of ten N, x, r where N is less than 15 bits and x and r are not trivial:

N	x	r
15	13	4
21	19	6
35	12	12
33	7	10
55	41	10
77	8	10
91	47	12
143	4	30
221	126	48
323	71	144

Period Finding Unitary Matrix

Write code to produce a period finding unitary matrix for a given co-prime (x,N). Give an example of an output unitary matrix:

```
def build_U(x, N):
    """Build the unitary matrix U|j> = |(j*x) mod N> for j<N, |j> for j>=N."""
    n = math.ceil(math.log2(N))
    dim = 2 ** n
    U = np.zeros((dim, dim), dtype=complex)

    for j in range(dim):
        if j < N:
            U[(j * x) % N, j] = 1
        else:
            U[j, j] = 1

    return U
```

For a few different examples of x, N, generate the matrix U and find its eigenvalues e. Also, compute the period r using your Classical Shor's algorithm. For each (x,N), paste the vector of eigenvalues e, the period r, and the vector e*r, which should be integers:

N = 15, x = 7, n_qubits = 4 (Matrix dim: 16x16)
True period r = 4

Index	Phase (theta)	theta * r	Integer?
0	0.000000	0.000000	0
1	0.000000	0.000000	0
2	0.000000	0.000000	0
3	0.000000	0.000000	0
4	0.000000	0.000000	0
5	0.000000	0.000000	0
6	0.000000	0.000000	0
7	0.250000	1.000000	1
8	0.250000	1.000000	1
9	0.250000	1.000000	1
10	0.500000	2.000000	2
11	0.500000	2.000000	2
12	0.500000	2.000000	2

13	0.750000	3.000000	3
14	0.750000	3.000000	3
15	0.750000	3.000000	3

Show that you can find the period r given a random eigenvalue of the matrix U for a particular (x,N).

Trial 1: theta = 0.250000 -> r = 4 OK
Trial 2: theta = 0.750000 -> r = 4 OK
Trial 3: theta = 0.250000 -> r = 4 OK
Trial 4: theta = 0.250000 -> r = 4 OK
Trial 5: theta = 0.250000 -> r = 4 OK

Using the above algorithm for factoring using only the eigenvalues of the U matrix (without the help of the Classical Shor's algorithm), factor some numbers. Paste the output here:

N = 15 -> 3 * 5 = 15 OK
N = 21 -> 3 * 7 = 21 OK
N = 35 -> 7 * 5 = 35 OK
N = 33 -> 11 * 3 = 33 OK
N = 55 -> 11 * 5 = 55 OK
N = 77 -> 11 * 7 = 77 OK
N = 91 -> 7 * 13 = 91 OK
N = 143 -> 13 * 11 = 143 OK
N = 221 -> 13 * 17 = 221 OK
N = 323 -> 19 * 17 = 323 OK
N = 989 -> 23 * 43 = 989 OK

Shor's Algorithm

Show that your quantum computing simulator running the Quantum Shor's circuit can successfully factor numbers. Try to factor 21.

15 -> 3 * 5 OK
21 -> 3 * 7 OK
35 -> 5 * 7 OK

Show that your simulator runs faster with the speed-up trick.

N	naive	fast	speedup
15	271	24	11x
21	1043	30	35x
33	4119	36	114x
35	4119	36	114x

Extra Credit:

Add below any additional extra credit that you've achieved